

Практическое задание 3

Механизмы межзадачного взаимодействия

<https://github.com/ananass12/Real-time-systems>

Классические подходы к IPC

1. **Выбор механизма IPC** зависит от требований:
 - Структурированные сообщения - POSIX MQ
 - Поточковые данные - UNIX Sockets
 - Максимальная производительность - Shared Memory (разделяемая память)
1. **Синхронизация** обязательна при совместном доступе к ресурсам
2. **Масштабируемость** достигается за счет асинхронных моделей (epoll)
3. **Производительность** оптимизируется через минимизацию копирований данных

Задание 1

Задание 1: POSIX Message Queues (`posix_mq_...`)

- **Цель:** Освоить стандартизированный, надежный, ориентированный на сообщения механизм IPC.
- **Действия:**
 1. Реализуйте пару программ: `posix_mq_server.c` и `posix_mq_client.c`.
 2. Клиент должен отправлять серверу строку, а сервер — преобразовывать ее в верхний регистр и отправлять обратно в другую очередь, которую слушает клиент.
 3. Используйте сообщения с разными приоритетами: срочные сообщения должны обрабатываться сервером раньше обычных.
 4. В комментариях к коду опишите плюсы и минусы MQ (например, ограничение на размер и кол-во сообщений vs. встроенные приоритеты и надежность).

POSIX Message Queues (MQ)

- один из способов межпроцессного взаимодействия (IPC) в Linux/Unix
- позволяет разным программам (процессам) обмениваться сообщениями через специальные очереди
- каждое сообщение может иметь приоритет
- сообщения не теряются, пока очередь существует.

POSIX Message Queues (MQ):

- **Суть:** Очередь структурированных сообщений, управляемая ядром.

- **Плюсы:** Гарантированная доставка, поддержка приоритетов сообщений, фиксированный размер очереди, простота синхронизации (не нужно самому реализовывать очереди и mutex'ы).
- **Минусы:** Работает только в пределах одной машины.
- **Сценарий использования:** Асинхронный обмен дискретными сообщениями или событиями между процессами на одном хосте, где важны порядок (в т.ч. по приоритету) и надежность (например, задача "Producer-Consumer").

Сервер: ждёт сообщения в /server_queue, обрабатывает текст (конвертирует в верхний регистр), возвращает через очередь клиента /client_queue.

Клиент: отправляет сообщения серверу, ждёт ответ, использует разные приоритеты.

Пример работы

1. Клиент отправляет: "ordinary message 1" (обычный приоритет) - сервер получает - делает "ORDINARY MESSAGE 1" - возвращает клиенту.
2. Клиент отправляет: "urgent message!" (высокий приоритет) - сервер получает раньше других обычных сообщений - возвращает "URGENT MESSAGE!".

```

bash: ./posix_mq_server: нет такого файла или каталога
nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/posix_mq_server
Server is running and waiting for messages...
Received message with priority 1: "ordinary message 1"
Sent answer: "ORDINARY MESSAGE 1"
Received message with priority 10: "urgent message!"
Sent answer: "URGENT MESSAGE!"
Received message with priority 1: "ordinary message 2"
Sent answer: "ORDINARY MESSAGE 2"

```

```

nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/posix_mq_client
Send message with priority 1: "ordinary message 1"
Received answer: "ORDINARY MESSAGE 1"

Send message with priority 10: "urgent message!"
Received answer: "URGENT MESSAGE!"

Send message with priority 1: "ordinary message 2"
Received answer: "ORDINARY MESSAGE 2"

```

Задание 2

Задание 2: Асинхронный сервер на `epoll` и `eventfd` (`epoll_server.c`)

- **Цель:** Построить каркас асинхронного сервера, способного эффективно управлять множеством клиентов и внутренними событиями.
- **Действия:**
 1. Создайте сервер, который слушает на UNIX-сокете.
 2. Используйте `epoll` для одновременного ожидания:
 - Новых подключений.
 - Данных от уже подключенных клиентов.
 - Внутренних событий от других потоков сервера через `eventfd`.
 3. Реализуйте корректное **обнаружение обрыва соединения** (клиент закрыл сокет). Сервер должен удалить файловый дескриптор клиента из `epoll` и очистить ресурсы.
 4. В комментариях к коду объясните, почему `epoll` масштабируется лучше, чем `poll` или `select` на большом количестве соединений.

epoll — механизм мониторинга множества файловых дескрипторов:

- **Level-Triggered (LT)** — уведомления пока есть данные, проще в использовании, подходит для большинства задач
- **Edge-Triggered (ET)** — уведомления только при изменении состояния, требует неблокирующих операций и цикла чтения/записи до EAGAIN

eventfd — легковесный дескриптор для уведомлений между потоками.

Асинхронный сервер на `epoll` + `eventfd`

- Это сервер, который умеет одновременно работать с **множеством клиентов** без блокировки.
- Использует **UNIX-сокеты**, чтобы клиенты могли подключаться к серверу через файловую систему.
- Основные механизмы:
 1. **epoll** — позволяет «наблюдать» сразу за множеством файловых дескрипторов (сокеты, `eventfd` и др.) и реагировать только на те, где есть событие.
 2. **eventfd** — внутренний сигнал для сервера (например, от другого потока) без использования сокета.

Как работает сервер

1. Создаёт UNIX-сокеты и слушает его (`listen`).
2. Создаёт `epoll`, добавляет сокет сервера и `eventfd` в список наблюдения.
3. В цикле ждёт события (`epoll_wait`):
 - **Новое подключение клиента** → добавляет его сокет в `epoll` для чтения.

- **Данные от клиента** → читает их и отправляет обратно (эхо).
- **Обрыв соединения** → клиент закрыл сокет → сервер закрывает fd и очищает ресурсы.
- **Внутреннее событие через eventfd** → сервер может реагировать на команды от других потоков.

Особенность	select/poll	epoll
Масштабируемость	При большом числе fd сильно падает	Работает быстро даже с тысячами fd
Проверка fd	Каждый раз проверяет все fd	События приходят только для готовых fd
Ресурсы	Постоянная копия fd в каждом вызове	fd добавляются один раз, потом события только для нужных
Асинхронность	Требуется многопоточность для масштабирования	Можно легко работать в одном потоке

- Набираем текст - сервер возвращает обратно (эхо).
- Закрываем socat - сервер корректно видит отключение.

```
nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/epoll_server
Server is listening on socket: /tmp/epoll_server.sock
Created eventfd, to emulate internal event execute:
echo 1 > /proc/13026/fd/5
```

```
New client (fd=6) connected.
Received from client (fd=6): hello
```

```
nastya@Ubuntu:~/RTS/tasks/task3$ socat - UNIX-CONNECT:/tmp/epoll_server.sock
hello
hello
```


Высокопроизводительный обмен данными

Задание 3

Задание 3: Векторный ввод-вывод (iov_demo.c)

- **Цель:** Изучить механизм scatter-gather I/O (`readv`/`writv`) для атомарной передачи структурированных данных без лишних копирований.
- **Действия:**
 1. Определите структуру, состоящую из нескольких частей (например, `struct { uint32_t msg_type; uint64_t msg_id; char payload[128]; }`).
 2. Заполните поля этой структуры.
 3. Создайте массив `struct iovec`, где каждый элемент указывает на одно из полей структуры.
 4. Используя `writv`, отправьте всю структуру в pipe одним системным вызовом.
 5. На читающей стороне аналогично подготовьте `iovec` и прочтите данные в поля структуры с помощью `readv`.
 6. В комментариях к коду объясните, в чем выгода от использования `writv` по сравнению с ручным `memcpy` всех частей в единый буфер перед отправкой.

Векторный ввод-вывод - механизм в операционных системах, позволяющий за один системный вызов выполнить операцию чтения или записи с использованием нескольких буферов памяти.

Основные системные вызовы:

- `writv()` — запись из нескольких буферов
- `readv()` — чтение в несколько буферов

Преимущества векторного I/O:

1. Снижение системных вызовов

- Вместо N вызовов `write()` — один вызов `writv()`
- Уменьшение накладных расходов на переключение в ядро

2. Атомарность операций

- Для pipes, FIFO, сокетов данные записываются как единое целое
- Исключает перемешивание с данными других процессов

iov_demo.c

Код демонстрирует векторный ввод-вывод в Linux с помощью системных вызовов `writv` и `readv`. Он показывает, как можно атомарно отправить или получить составное сообщение, состоящее из нескольких непрерывных, но физически отдельных участков памяти, без предварительного копирования всех частей в один буфер.

`iovec` описывает один буфер для операций ввода-вывода. Массив таких структур позволяет работать с несколькими несмежными буферами за одну системную операцию.

Принцип работы:

1. Создание pipe (`pipe_fd[0]` для чтения, `pipe_fd[1]` для записи)
2. Определение структуры `message_t` с полями `msg_type`, `msg_id`, `payload`
3. Настройка массива `iovec` для отправки всех частей структуры одним вызовом `writv()`
4. Чтение с помощью `readv()`

Преимущество: удобство передачи структурированных данных и экономия времени.

Почему полезно использовать `writv/readv`:

- Нет необходимости делать `memcpy()` всех частей в один большой буфер.
- Атомарная передача нескольких буферов — экономия ресурсов и времени.
- Удобно для структурированных данных (например, пакеты в сети).

```

make: [makefile.15: bin/shm_consumer] oshmkd 1
● nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/iov_demo
--- WRITER ---
Preparing to send:
  msg_type: 1
  msg_id:   9988776655443322
  payload:  "Hello, IOV!"
writev() wrote 23 bytes.

--- READER ---
readv() read 23 bytes.
Received data:
  msg_type: 1
  msg_id:   9988776655443322
  payload:  "Hello, IOV!"

Check passed: data matches.
○ nastya@Ubuntu:~/RTS/tasks/task3$

```

Задание 4

Задание 4: Общая память и синхронизация (shm_...)

- **Цель:** Реализовать самый быстрый способ IPC через общую память и понять критическую важность ее синхронизации.
- **Действия:**
 1. Создайте общую для программ структуру в заголовочном файле `shm_common.h`.
 2. Напишите `shm_producer.c`: он создает или открывает сегмент POSIX shared memory (`shm_open`, `mmap`) и начинает циклически записывать в него данные (например, увеличивающийся счетчик).
 3. Напишите `shm_consumer.c`: он открывает тот же сегмент и читает из него данные.
 4. **Сначала запустите их без синхронизации.** Пронаблюдайте и зафиксируйте в отчете артефакты (пропуски, "рванные" чтения).
 5. **Добавьте синхронизацию:** используйте **именованные семафоры POSIX** (`sem_open`, `sem_wait`, `sem_post`), чтобы производитель и потребитель обращались к общей памяти по очереди.
 6. Убедитесь, что с семафорами данные передаются корректно. Сравните в комментариях к коду оба запуска.

Семафоры — примитив синхронизации для управления доступом к разделяемым ресурсам:

- Счетчик доступных ресурсов
- Атомарные операции wait/post
- Совместное использование между процессами

Запуск без семафоров:

- убрала usleep в producer
- Уменьшить BUFFER_SIZE до 3

Проблемы без синхронизации:

- Состояние гонки (race condition)
- Потеря обновлений данных
- Несогласованность состояния

```
● nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/shm_producer
Producer: Shared memory segment created and mapped.
Producer: Semaphores created.
Produced: 0 at index 0
Produced: 1 at index 1
Produced: 2 at index 2
Produced: 3 at index 3
Produced: 4 at index 4
Produced: 5 at index 5
Produced: 6 at index 6
Produced: 7 at index 7
Produced: 8 at index 8
Produced: 9 at index 9
Produced: 10 at index 0
Produced: 11 at index 1
Produced: 12 at index 2
Produced: 13 at index 3
Produced: 14 at index 4
Produced: 15 at index 5
Produced: 16 at index 6
Produced: 17 at index 7
Produced: 18 at index 8
Produced: 19 at index 9
```



```
o nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/shm_consumer
Consumer: Shared memory segment opened and mapped.
Consumer: Semaphores opened.
Consumed: 50 from index 0
Consumed: 51 from index 1
Consumed: 52 from index 2
Consumed: 53 from index 3
Consumed: 54 from index 4
Consumed: 55 from index 5
Consumed: 56 from index 6
Consumed: 57 from index 7
Consumed: 58 from index 8
Consumed: 59 from index 9
Consumed: 70 from index 0
Consumed: 71 from index 1
Consumed: 72 from index 2
Consumed: 73 from index 3
Consumed: 74 from index 4
Consumed: 75 from index 5
Consumed: 76 from index 6
Consumed: 77 from index 7
Consumed: 78 from index 8
Consumed: 79 from index 9
Consumed: 90 from index 0
Consumed: 91 from index 1
```

```
Consumer: Resources freed.
o nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/shm_consumer
Consumer: Shared memory segment opened and mapped.
Consumer: Semaphores opened.
Consumed: 917793 from index 0
Consumed: 945592 from index 1
Consumed: 988589 from index 2
Consumed: 1023921 from index 0
Consumed: 1061251 from index 1
Consumed: 1097723 from index 2
Consumed: 1137927 from index 0
Consumed: 1175644 from index 1
Consumed: 1214978 from index 2
Consumed: 1252305 from index 0
```

Запуск с семафорами:

всё работает корректно, даже при высокой скорости записи.

```
nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/shm_producer
```

```
Produced: 68 at index 8  
Produced: 69 at index 9  
Produced: 70 at index 0  
Produced: 71 at index 1  
Produced: 72 at index 2  
Produced: 73 at index 3  
Produced: 74 at index 4  
Produced: 75 at index 5  
Produced: 76 at index 6  
Produced: 77 at index 7  
Produced: 78 at index 8  
Produced: 79 at index 9  
^CProduced: 80 at index 0
```

```
Producer: End of work...  
Producer: Resources freed.
```

```
nastya@Ubuntu:~/RTS/tasks/task3$
```

```
nastya@Ubuntu:~/RTS/tasks/task3$ ./bin/shm_consumer
```

```
Consumed: 65 from index 5  
Consumed: 66 from index 6  
Consumed: 67 from index 7  
Consumed: 68 from index 8  
Consumed: 69 from index 9  
Consumed: 80 from index 0  
Consumed: 71 from index 1  
Consumed: 72 from index 2  
Consumed: 73 from index 3  
Consumed: 74 from index 4  
Consumed: 75 from index 5  
Consumed: 76 from index 6  
Consumed: 77 from index 7  
Consumed: 78 from index 8  
Consumed: 79 from index 9  
Consumed: 80 from index 0
```

```
Producer: Resources freed.  
○ nasty@Ubuntu:~/RTS/tasks/task3$ ./bin/shm_producer  
Producer: Shared memory segment created and mapped.  
Producer: Semaphores created.  
Produced: 0 at index 0  
Produced: 1 at index 1  
Produced: 2 at index 2  
Produced: 3 at index 3  
Produced: 4 at index 4  
Produced: 5 at index 5  
Produced: 6 at index 6  
Produced: 7 at index 7  
Produced: 8 at index 8  
Produced: 9 at index 9  
Produced: 10 at index 0  
Produced: 11 at index 1  
Produced: 12 at index 2  
Produced: 13 at index 3
```

```
○ nasty@Ubuntu:~/RTS/tasks/task3$ ./bin/shm_consumer  
Consumer: Shared memory segment opened and mapped.  
Consumer: Semaphores opened.  
Consumed: 0 from index 0  
Consumed: 1 from index 1  
Consumed: 2 from index 2  
Consumed: 3 from index 3  
Consumed: 4 from index 4  
Consumed: 5 from index 5  
Consumed: 6 from index 6  
Consumed: 7 from index 7  
Consumed: 8 from index 8  
Consumed: 9 from index 9  
Consumed: 10 from index 0  
Consumed: 11 from index 1  
Consumed: 12 from index 2  
Consumed: 13 from index 3
```

Вывод: семафоры необходимы для синхронизации при быстром обмене через общую память.

Контрольные вопросы

1. **POSIX MQ vs. UNIX Sockets:** В каких случаях вы бы предпочли использовать очередь сообщений, а в каких — сокеты? Опишите по одному сценарию для каждого.
2. **Edge-Triggered (ET) vs. Level-Triggered (LT) в `epoll`:** Опишите разницу в поведении `epoll` в режимах `EPOLLET` и `EPOLLIN` (по умолчанию). Какой режим сложнее в использовании и почему? Какие ошибки можно допустить при работе с ET?
3. **Семафоры vs. Мьютексы:** В задании 4 мы использовали семафоры. Можно ли было использовать мьютекс (`pthread_mutex_t`) для синхронизации доступа к общей памяти между разными процессами? Объясните, почему да или нет, и какие атрибуты мьютекса для этого потребовались бы.
4. **Копирование данных ядром:** Расположите изученные механизмы (MQ, UNIX Sockets, Shared Memory) в порядке возрастания количества копирований данных между ядром и пользовательским пространством при передаче. Объясните свой ответ.

1. POSIX MQ vs. UNIX Sockets

POSIX Message Queue

- Используется для асинхронного обмена сообщениями между процессами на одной системе.
- Плюсы: гарантированная доставка сообщений, приоритеты, очередь фиксированного размера, проще синхронизировать.
- Сценарий: процессы A и B внутри одной системы обмениваются событиями (например, producer-consumer), где важно, чтобы сообщения не терялись и обрабатывались по порядку.

UNIX Sockets (SOCK_STREAM или SOCK_DGRAM)

- Подходит для сетевого или локального взаимодействия, можно между машинами.
- Плюсы: потоковая передача, можно строить клиент-серверные приложения, поддержка TCP/UDP.
- Сценарий: локальный сервер логов принимает сообщения от нескольких клиентских процессов через сокеты (можно и TCP, и UNIX domain sockets), нужен поток данных, а не фиксированные сообщения.

Вывод: MQ удобнее для асинхронного, структурированного обмена на одной машине; сокеты — для потоковой передачи и сетевого взаимодействия.

2. Edge-Triggered (ET) vs. Level-Triggered (LT) в `epoll`

Режим	Поведение	Пример
Level-Triggered (LT, по умолчанию, <code>EPOLLIN</code>)	<code>epoll_wait</code> возвращает событие, пока есть данные в <code>fd</code> .	Если данные не прочитаны полностью, событие будет возвращаться снова и снова.
Edge-Triggered (ET, <code>EPOLLET</code>)	<code>epoll_wait</code> возвращает событие только при изменении состояния <code>fd</code> (например, приход новых данных).	Нужно читать все доступные данные сразу, иначе события могут быть пропущены.

Особенности ET:

- Сложнее в использовании, потому что нужно реализовать цикл чтения/записи до EAGAIN, иначе можно пропустить события.
- Типичные ошибки:
 - о Читать только один байт вместо всего доступного → данные “зависнут” без новых уведомлений.
 - о Не учитывать частичное чтение/запись буфера.

3. Семафоры vs. Мьютексы

Можно ли использовать мьютекс (pthread_mutex_t) между процессами?

Да, но нужно включить pshared атрибут.

Стандартный мьютекс синхронизирует потоки внутри одного процесса.

Для синхронизации между процессами нужен атрибут PTHREAD_PROCESS_SHARED, чтобы мьютекс был разделяемым в памяти, например, в shared memory.

Сравнение с семафорами: семафор проще, не требует дополнительной инициализации атрибутов, сразу работает между процессами.

4. Копирование данных ядром

Механизмы передачи и количество копирований:

Механизм	Копирования данных (ядро <-> user space)
Shared Memory	0 копирований (процесс напрямую читает/пишет общую память)
POSIX MQ	1 копирование: пользователь - ядро (producer) и ядро - пользователь (consumer)
UNIX Sockets	2 копирования: пользователь - ядро - ядро - пользователь (при отправке через сокет TCP/UDP)

- Shared Memory — данные пишутся напрямую в область, видимую обоим процессам.
- Message Queue — данные копируются в очередь ядра и оттуда в другой процесс.
- Sockets — данные сначала копируются в ядро, потом ядро пересылает и копирует в память получателя.